Lab7

Name : Sanjana

SRN : PES1UG24CS420

Section : G

Buggy code

```
C:\Users\Admin\Desktop\L6_L7_55_car_insurance_premium\LAB7>python car_insurance_premium_parser.py
==== Running normalize_registration ====
Enter a registration number (e.g. ka 01 ab 1234): ka 01 ab 1234
Normalized registration: KA01AB1234

==== Running total_claim_amount ====
Enter past claim amounts separated by ; (e.g. 12000;4500;800): 12000;abc;800
Traceback (most recent call last):
  File "C:\Users\Admin\Desktop\L6_L7_55_car_insurance_premium\LAB7\car_insurance_premium_parser.py", line 39, in <module>
    main()
    ~~~~^^
  File "C:\Users\Admin\Desktop\L6_L7_55_car_insurance_premium\LAB7\car_insurance_premium_parser.py", line 32, in main
    print("Total claimed:", total_claim_amount(value))
                            ~~~~~~~~~~~~~~~~~~~^^^^^^^
  File "C:\Users\Admin\Desktop\L6_L7_55_car_insurance_premium\LAB7\car_insurance_premium_parser.py", line 16, in total_claim_amount
    return sum(int(amount) for amount in text.split(';'))
  File "C:\Users\Admin\Desktop\L6_L7_55_car_insurance_premium\LAB7\car_insurance_premium_parser.py", line 16, in <genexpr>
    return sum(int(amount) for amount in text.split(';'))
               ~~~^^^^^^^^
ValueError: invalid literal for int() with base 10: 'abc'

C:\Users\Admin\Desktop\L6_L7_55_car_insurance_premium\LAB7>
```

```
C:\Users\Admin\Desktop\L6_L7_55_car_insurance_premium\LAB7>python car_insurance_premium_parser.py
==== Running normalize_registration ====
Enter a registration number (e.g. ka 01 ab 1234): ka 01 ab 1234
Normalized registration: KA01AB1234

==== Running total_claim_amount ====
Enter past claim amounts separated by ; (e.g. 12000;4500;800): 12000;4500;800
Total claimed: 17300

==== Running driver_age ====
Enter the driver's date of birth as DD/MM/YYYY (e.g. 14/08/1999): 14-08-1999
Traceback (most recent call last):
  File "C:\Users\Admin\Desktop\L6_L7_55_car_insurance_premium\LAB7\car_insurance_premium_parser.py", line 39, in <module>
    main()
    ~~~~^^
  File "C:\Users\Admin\Desktop\L6_L7_55_car_insurance_premium\LAB7\car_insurance_premium_parser.py", line 36, in main
    print("Driver age in 2026:", driver_age(value))
                                 ~~~~~~~~~~^^^^^^^
  File "C:\Users\Admin\Desktop\L6_L7_55_car_insurance_premium\LAB7\car_insurance_premium_parser.py", line 23, in driver_age
    return current_year - int(dob.split('/')[2])
                          ~~~~~~~~~~~~~~~~~~^^^
IndexError: list index out of range

C:\Users\Admin\Desktop\L6_L7_55_car_insurance_premium\LAB7>
```

Hypothesis tests failing on the buggy module.

```
C:\Users\Admin\Desktop\L6_L7_55_car_insurance_premium\LAB7>python -m pytest test_car_insurance_premium_parser.py
============================= test session starts =============================
platform win32 -- Python 3.13.7, pytest-9.2.0, pluggy-1.7.0
rootdir: C:\Users\Admin\Desktop\L6_L7_55_car_insurance_premium\LAB7
plugins: anyio-4.12.1, hypothesis-6.170.15, langsmith-0.6.6
collected 3 items

test_car_insurance_premium_parser.py FFF                                 [100%]

================================== FAILURES ===================================
_____________________ test_normalize_registration_no_crash _____________________

s = None

    @given(st.text() | st.none())
    def test_normalize_registration_no_crash(s):
        try:
>           normalize_registration(s)

test_car_insurance_premium_parser.py:7:
_ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _

plate = None

    def normalize_registration(plate):
        """
        Removes spaces and dashes from a registration number and upper-cases it.
        Assumes plate is a non-empty string.
        """
>       plate = plate.strip().upper()
                ^^^^^^^^^^^^
E       AttributeError: 'NoneType' object has no attribute 'strip'

car_insurance_premium_parser.py:6: AttributeError

During handling of the above exception, another exception occurred:

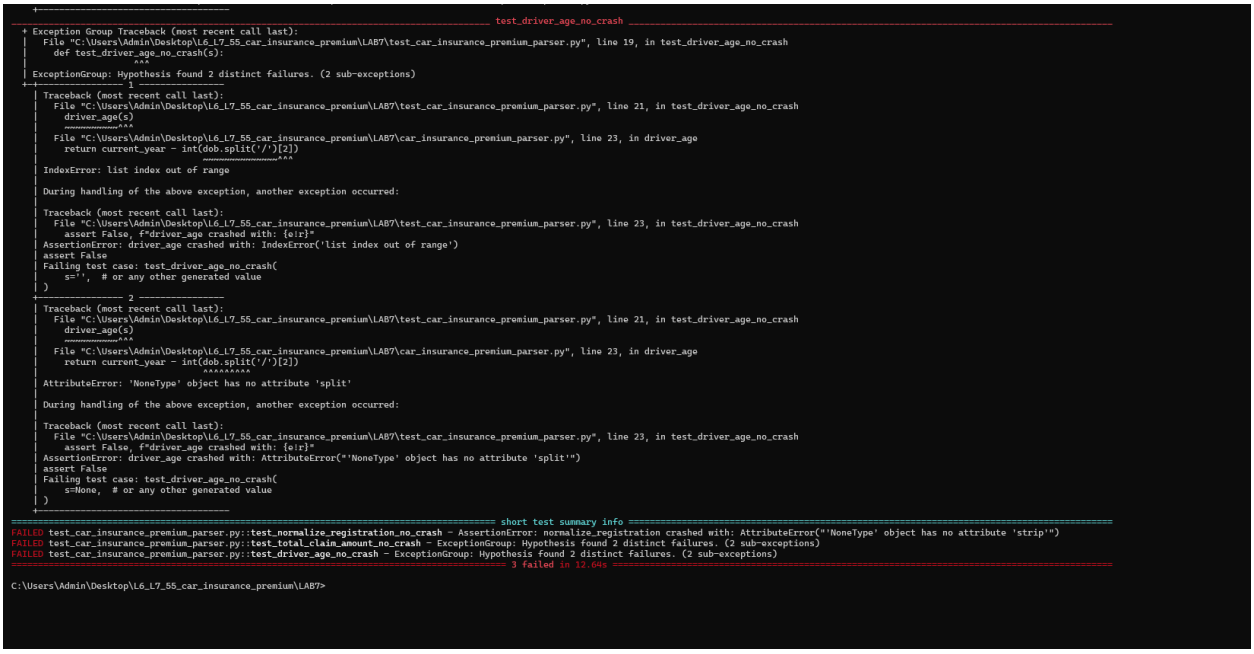
    @given(st.text() | st.none())
>   def test_normalize_registration_no_crash(s):
                ^^^

test_car_insurance_premium_parser.py:5:
_ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _

s = None

    @given(st.text() | st.none())
    def test_normalize_registration_no_crash(s):
        try:
            normalize_registration(s)
        except Exception as e:
>           assert False, f"normalize_registration crashed with: {e!r}"
E           AssertionError: normalize_registration crashed with: AttributeError("'NoneType' object has no attribute 'strip'")
E           assert False
E           Failing test case: test_normalize_registration_no_crash(
E               s=None,
E           )
E           Explanation:
E               These lines were always and only run by failing test cases:
E                   C:\Users\Admin\Desktop\L6_L7_55_car_insurance_premium\LAB7\test_car_insurance_premium_parser.py:8

test_car_insurance_premium_parser.py:9: AssertionError
_____________________ test_total_claim_amount_no_crash _____________________
  + Exception Group Traceback (most recent call last):
  |   File "C:\Users\Admin\Desktop\L6_L7_55_car_insurance_premium\LAB7\test_car_insurance_premium_parser.py", line 12, in test_total_claim_amount_no_crash
  |     def test_total_claim_amount_no_crash(s):
  |                ^^^
  | ExceptionGroup: Hypothesis found 2 distinct failures. (2 sub-exceptions)
  +-+---------------- 1 ----------------
    | Traceback (most recent call last):
    |   File "C:\Users\Admin\Desktop\L6_L7_55_car_insurance_premium\LAB7\test_car_insurance_premium_parser.py", line 14, in test_total_claim_amount_no_crash
    |     total_claim_amount(s)
    |     ~~~~~~~~~~~~~~~~~~~^^^
    |   File "C:\Users\Admin\Desktop\L6_L7_55_car_insurance_premium\LAB7\car_insurance_premium_parser.py", line 16, in total_claim_amount
    |     return sum(int(amount) for amount in text.split(';'))
    |   File "C:\Users\Admin\Desktop\L6_L7_55_car_insurance_premium\LAB7\car_insurance_premium_parser.py", line 16, in <genexpr>
    |     return sum(int(amount) for amount in text.split(';'))
    |                ~~~^^^^^^^^
    | ValueError: invalid literal for int() with base 10: ''
    |
    | During handling of the above exception, another exception occurred:
    |
    | Traceback (most recent call last):
    |   File "C:\Users\Admin\Desktop\L6_L7_55_car_insurance_premium\LAB7\test_car_insurance_premium_parser.py", line 16, in test_total_claim_amount_no_crash
    |     assert False, f"total_claim_amount crashed with: {e!r}"
    | AssertionError: total_claim_amount crashed with: ValueError("invalid literal for int() with base 10: ''")
    | assert False
    | Failing test case: test_total_claim_amount_no_crash(
    |     s='',
    | )
    | Explanation:
    |     These lines were always and only run by failing test cases:
    |         C:\Users\Admin\Desktop\L6_L7_55_car_insurance_premium\LAB7\test_car_insurance_premium_parser.py:15
    +---------------- 2 ----------------
    | Traceback (most recent call last):
    |   File "C:\Users\Admin\Desktop\L6_L7_55_car_insurance_premium\LAB7\test_car_insurance_premium_parser.py", line 14, in test_total_claim_amount_no_crash
    |     total_claim_amount(s)
    |     ~~~~~~~~~~~~~~~~~~~^^^
    |   File "C:\Users\Admin\Desktop\L6_L7_55_car_insurance_premium\LAB7\car_insurance_premium_parser.py", line 16, in total_claim_amount
    |     return sum(int(amount) for amount in text.split(';'))
    |                                          ^^^^^^^^^^
    | AttributeError: 'NoneType' object has no attribute 'split'
    |
    | During handling of the above exception, another exception occurred:
    |
    | Traceback (most recent call last):
    |   File "C:\Users\Admin\Desktop\L6_L7_55_car_insurance_premium\LAB7\test_car_insurance_premium_parser.py", line 16, in test_total_claim_amount_no_crash
    |     assert False, f"total_claim_amount crashed with: {e!r}"
    | AssertionError: total_claim_amount crashed with: AttributeError("'NoneType' object has no attribute 'split'")
    | assert False
    | Failing test case: test_total_claim_amount_no_crash(
    |     s=None,  # or any other generated value
    | )
    | Explanation:
    |     These lines were always and only run by failing test cases:
    |         C:\Users\Admin\Desktop\L6_L7_55_car_insurance_premium\LAB7\test_car_insurance_premium_parser.py:15
    +------------------------------------
_____________________ test_driver_age_no_crash _____________________
  + Exception Group Traceback (most recent call last):
  |   File "C:\Users\Admin\Desktop\L6_L7_55_car_insurance_premium\LAB7\test_car_insurance_premium_parser.py", line 19, in test_driver_age_no_crash
  |     def test_driver_age_no_crash(s):
  |                ^^^
  | ExceptionGroup: Hypothesis found 2 distinct failures. (2 sub-exceptions)
  +-+---------------- 1 ----------------
    | Traceback (most recent call last):
    |   File "C:\Users\Admin\Desktop\L6_L7_55_car_insurance_premium\LAB7\test_car_insurance_premium_parser.py", line 21, in test_driver_age_no_crash
    |     driver_age(s)
    |     ~~~~~~~~~~^^^
    |   File "C:\Users\Admin\Desktop\L6_L7_55_car_insurance_premium\LAB7\car_insurance_premium_parser.py", line 23, in driver_age
    |     return current_year - int(dob.split('/')[2])
    |                           ~~~~~~~~~~~~~~~~~~^^^
```



fixed module run

```
C:\Users\Admin\Desktop\L6_L7_55_car_insurance_premium\LAB7>python car_insurance_premium_parser.py
==== Running normalize_registration ====
Enter a registration number (e.g. ka 01 ab 1234): ka 01 ab 1234
Normalized registration: KA01AB1234

==== Running total_claim_amount ====
Enter past claim amounts separated by ; (e.g. 12000;4500;800): 12000;abc;800
Total claimed: 12800

==== Running driver_age ====
Enter the driver's date of birth as DD/MM/YYYY (e.g. 14/08/1999): 14/08/1999
Driver age in 2026: 27

C:\Users\Admin\Desktop\L6_L7_55_car_insurance_premium\LAB7>python car_insurance_premium_parser.py
==== Running normalize_registration ====
Enter a registration number (e.g. ka 01 ab 1234): ka 01 ab 1234
Normalized registration: KA01AB1234

==== Running total_claim_amount ====
Enter past claim amounts separated by ; (e.g. 12000;4500;800): 12000;4500;800
Total claimed: 17300

==== Running driver_age ====
Enter the driver's date of birth as DD/MM/YYYY (e.g. 14/08/1999): 14-08-1999
Invalid date of birth. Please use DD/MM/YYYY.

C:\Users\Admin\Desktop\L6_L7_55_car_insurance_premium\LAB7>
```

Hypothesis tests passing on the fixed module.

```
C:\Users\Admin\Desktop\L6_L7_55_car_insurance_premium\LAB7>python -m pytest test_car_insurance_premium_parser.py
test session starts
platform win32 -- Python 3.13.7, pytest-9.2.0, pluggy-1.7.0
rootdir: C:\Users\Admin\Desktop\L6_L7_55_car_insurance_premium\LAB7
plugins: anyio-4.12.1, hypothesis-6.170.15, langsmith-0.6.6
collected 3 items

test_car_insurance_premium_parser.py ...                                 [100%]

3 passed in 5.05s

C:\Users\Admin\Desktop\L6_L7_55_car_insurance_premium\LAB7>
```

Bugs found and how each was fixed

Function	Failing input found by Hypothesis	Error	Fix
normalize_registration	`None`	AttributeError (`None` has no `.strip`)	Return `""` if the input isn't a string
total_claim_amount	`None`, `""`, `"abc"`, `"1;;2"`, `"12.5"`	AttributeError / ValueError	Return 0 for non-strings; skip empty, non-numeric and negative amounts (ASCII digits only)
driver_age	`None`, `""`, `"14-08-1999"`	AttributeError / IndexError	Validate it's a string with exactly 3 `/`-separated digit parts; check it's a real date (so `31/02/2000` is rejected) and not in the future; return `None` if invalid

car_insurance_premium_parser

```
from datetime import date


def normalize_registration(plate):
    """
    Removes spaces, dashes and dots from a registration number and upper-cases
it.
    Returns an empty string if plate is not a string (e.g. None).
    """
    if not isinstance(plate, str):
        return ""
    plate = plate.strip().upper()
    for ch in [' ', '-', '.']:
        plate = plate.replace(ch, '')
    return plate


def total_claim_amount(text):
    """
    Adds up claim amounts separated by semicolons.
    Empty, non-numeric, negative or malformed amounts are ignored.
    Returns 0 if text is not a string (e.g. None).
```

```
    """
    if not isinstance(text, str):
        return 0
    total = 0
    for amount in text.split(';'):
        amount = amount.strip()
        # accept only plain ASCII digits (rejects "abc", "12.5", "-5", "",
"1_000")
        if not (amount.isascii() and amount.isdigit()):
            continue
        try:
            total += int(amount)
        except ValueError:
            # e.g. absurdly long digit strings exceeding Python's int limit
            continue
    return total


def driver_age(dob, current_year=2026):
    """
    Works out the driver's age from a DD/MM/YYYY date of birth.
    Returns None if dob is not a string, is not in DD/MM/YYYY format,
    is not a real calendar date, or is in the future.
    """
    if not isinstance(dob, str):
        return None
    parts = dob.strip().split('/')
    if len(parts) != 3:
        return None
    for p in parts:
        p = p.strip()
        if not (p.isascii() and p.isdigit()):
            return None
    try:
        day, month, year = (int(p.strip()) for p in parts)
        date(year, month, day)  # raises ValueError for impossible dates
    except ValueError:
        return None
    if year > current_year:
        return None
    return current_year - year


def main():
    print("==== Running normalize_registration ====")
```

```
    value = input("Enter a registration number (e.g. ka 01 ab 1234): ")
    print("Normalized registration:", normalize_registration(value))

    print("\n==== Running total_claim_amount ====")
    value = input("Enter past claim amounts separated by ; (e.g. 12000;4500;800):
")
    print("Total claimed:", total_claim_amount(value))

    print("\n==== Running driver_age ====")
    value = input("Enter the driver's date of birth as DD/MM/YYYY (e.g.
14/08/1999): ")
    age = driver_age(value)
    if age is None:
        print("Invalid date of birth. Please use DD/MM/YYYY.")
    else:
        print("Driver age in 2026:", age)

if __name__ == "__main__":
    main()
```

test_car_insurance_premium_parser

```
from hypothesis import given, strategies as st
from car_insurance_premium_parser import normalize_registration,
total_claim_amount, driver_age

@given(st.text() | st.none())
def test_normalize_registration_no_crash(s):
    try:
        normalize_registration(s)
    except Exception as e:
        assert False, f"normalize_registration crashed with: {e!r}"

@given(st.text() | st.none())
def test_total_claim_amount_no_crash(s):
    try:
        total_claim_amount(s)
    except Exception as e:
        assert False, f"total_claim_amount crashed with: {e!r}"

@given(st.text() | st.none())
def test_driver_age_no_crash(s):
    try:
```

```
        driver_age(s)
    except Exception as e:
        assert False, f"driver_age crashed with: {e!r}"
```